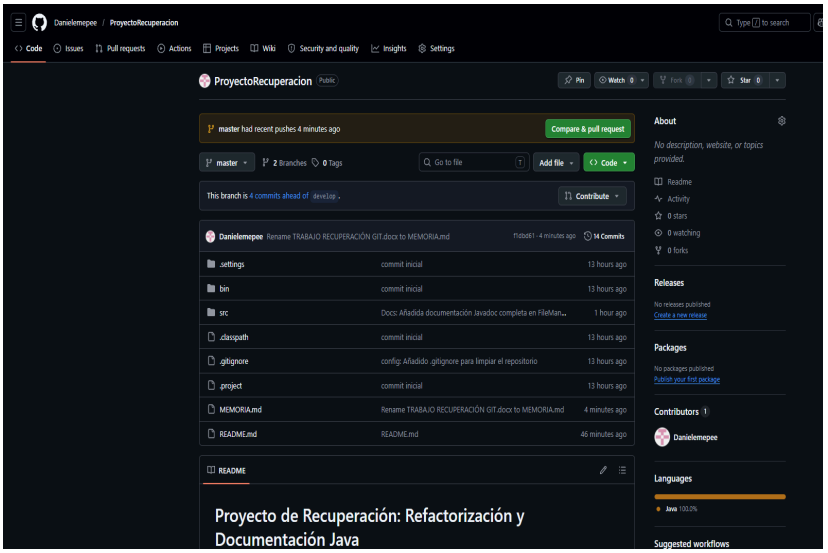
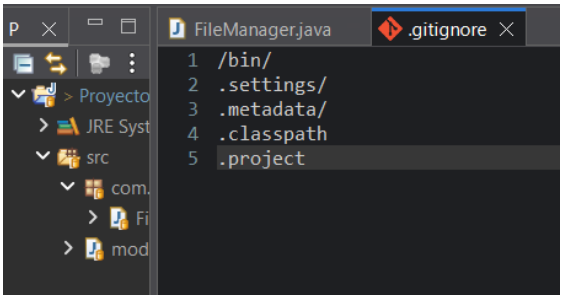
TRABAJO RECUPERACIÓN GIT

Creación del repositorio:

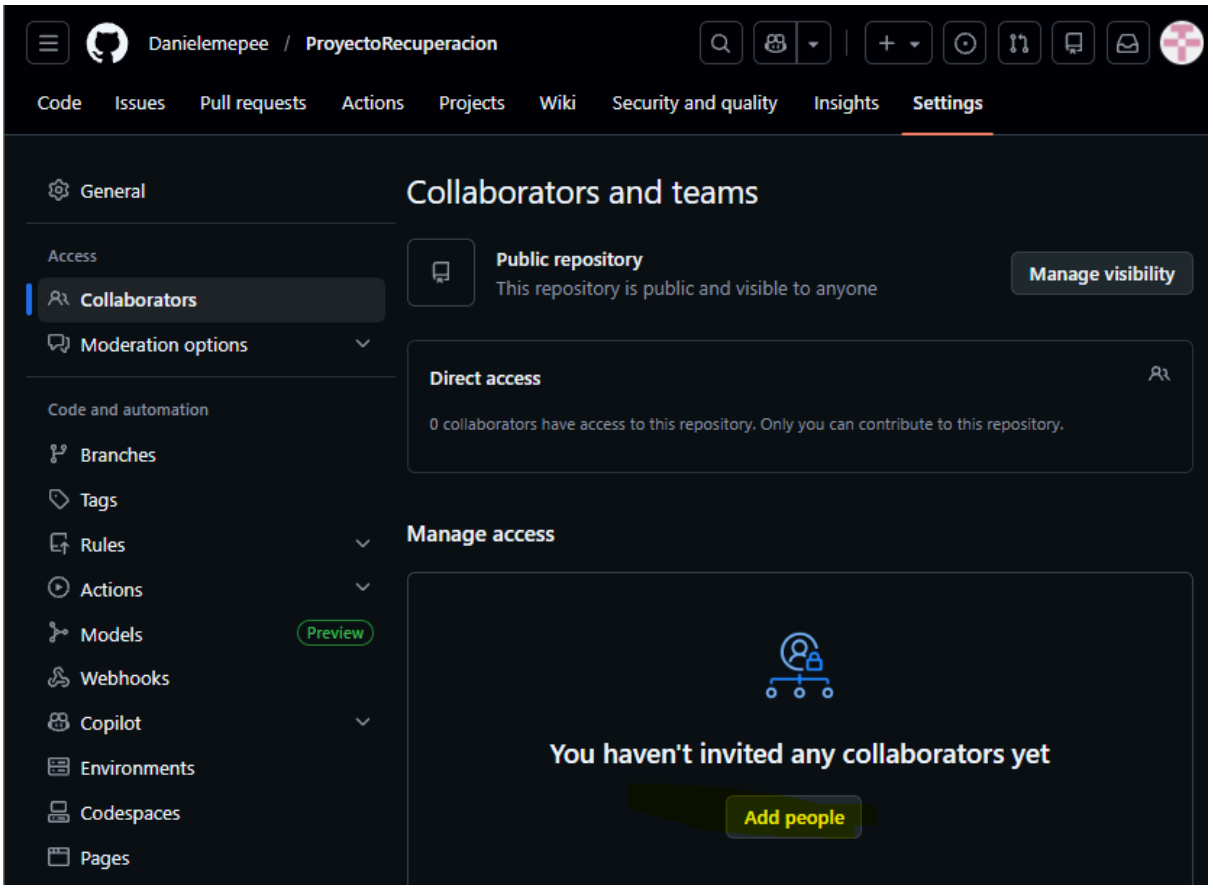


Creo el **.gitignore** para evitar subir archivos innecesarios.



Creo el .README. Es el documento que muestra lapágina del repositorio. Conseguimos que cualquier persona entienda el proyecto

AÑADIR COLABORADORES *



Protejo master

Branch name pattern *

master

Protect matching branches

- [x] **Require a pull request before merging**
  When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

  - [x] **Require approvals**
    When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

    Required number of approvals before merging: 1

2. Refactorizaciones al código

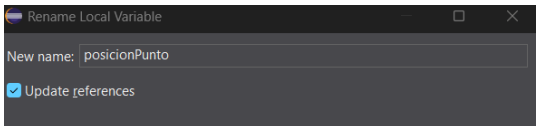
2.1 Renombrar variable

- **Descripción:** Se ha modificado el nombre de la variable local i por un nombre más representativo: posicionPunto. He utilizado la herramienta de refactorización de Eclipse (click derecho-> Refactor-> Rename).
- **Justificación:** El uso de nombres de una sola letra es una mala práctica que dificulta la legibilidad. Al usar posicionPunto, el código se vuelve autodocumentado,haciendo que cualquier desarrollador entienda qué información almacena esa variable.

Antes

```java
package com.recuperacion;

import java.io.File;

public class FileManager {

    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamano = archivo.length();
            System.out.println("Tamaño: " + tamano + " bytes");

            if (tamano > 1024 * 1024) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = "";
            int i = ruta.lastIndexOf('.');
            if (i > 0) {
                extension = ruta.substring(i + 1);
            }
            System.out.println("Extensión: " + extension);
        }
    }
}
```

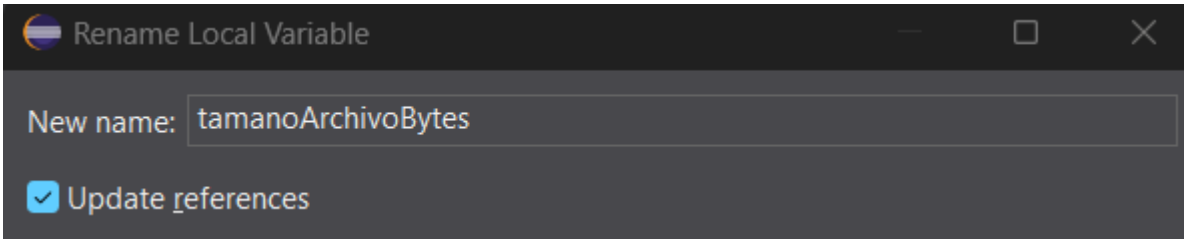
Después




```java
package com.recuperacion;

import java.io.File;

public class FileManager {

    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamano = archivo.length();
            System.out.println("Tamaño: " + tamano + " bytes");

            if (tamano > 1024 * 1024) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = "";
            int posicionPunto = ruta.lastIndexOf('.');
            if (posicionPunto > 0) {
                extension = ruta.substring(posicionPunto + 1);
            }
            System.out.println("Extensión: " + extension);
        }
    }
}
```

- 2a Variable. Se han modificado los nombres de variables por nombres descriptivos.

He modificado el nombre de la variable “tamano” por “tamanoArchivoBytes” para que sea más descriptivo




```java
    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamanoArchivoBytes = archivo.length();
            System.out.println("Tamaño: " + tamanoArchivoBytes + " bytes");

            if (tamanoArchivoBytes > TAMANO_MAX_BYTES) {
                System.out.println("Archivo grande, comprimiendo...");
            }
```

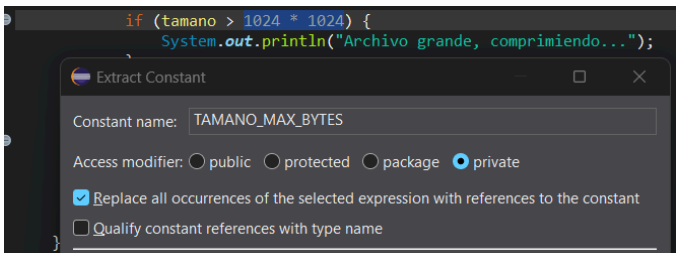
2.2 Extraer Constante

- **Descripción:** Se ha identificado el valor numérico 1024 * 1024 y se ha extraído a una constante estática y final: TAMANO_MAXIMO_BYTES.
- **Justificación:** Esta refactorización elimina lo que se conoce como "Números Mágicos" . Al centralizar el valor en una constante, facilitamos futuros cambios y evitamos errores si ese valor se repitiera en otras partes del programa.

<u>Antes</u>

```java
package com.recuperacion;

import java.io.File;

public class FileManager {

    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamano = archivo.length();
            System.out.println("Tamaño: " + tamano + " bytes");

            if (tamano > 1024 * 1024) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = "";
            int posicionPunto = ruta.lastIndexOf('.');
            if (posicionPunto > 0) {
                extension = ruta.substring(posicionPunto + 1);
            }
            System.out.println("Extensión: " + extension);
        }
    }
}
```

<u>Después</u>



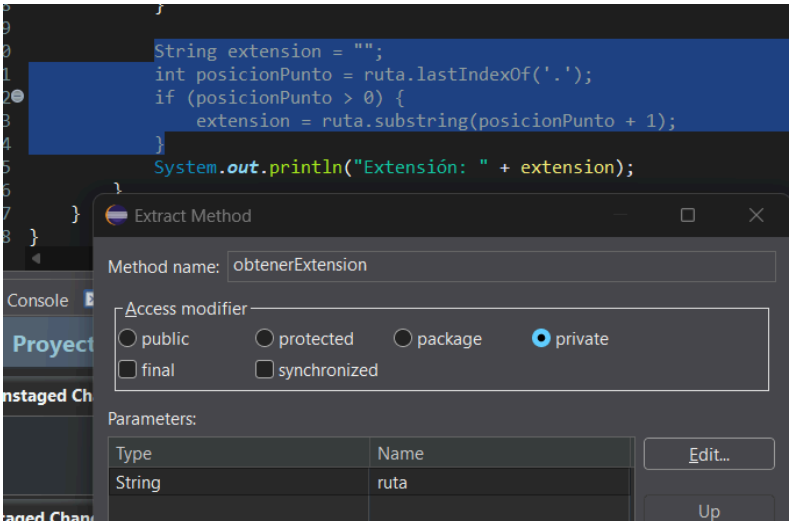

```java
private static final int TAMANO_MAX_BYTES = 1024 * 1024;

public void procesarArchivo(String ruta) {
    File archivo = new File(ruta);
    if (archivo.exists()) {
        System.out.println("Archivo encontrado: " + archivo.getName());
        long tamano = archivo.length();
        System.out.println("Tamaño: " + tamano + " bytes");

        if (tamano > TAMANO_MAX_BYTES) {
            System.out.println("Archivo grande, comprimiendo...");
        }

        String extension = "";
        int posicionPunto = ruta.lastIndexOf('.');
        if (posicionPunto > 0) {
            extension = ruta.substring(posicionPunto + 1);
        }
        System.out.println("Extensión: " + extension);
```

2.3. Extraer método

Descripción: Se ha tomado el bloque de código encargado de procesar la cadena de texto para obtener la extensión del archivo y se ha movido a un nuevo método privado llamado `obtenerExtension`.

Justificación: El método original `procesarArchivo` estaba haciendo demasiadas cosas (gestionar el archivo, comprobar el tamaño y calcular la extensión). Al extraer la lógica de la extensión, el código es más modular.

Antes

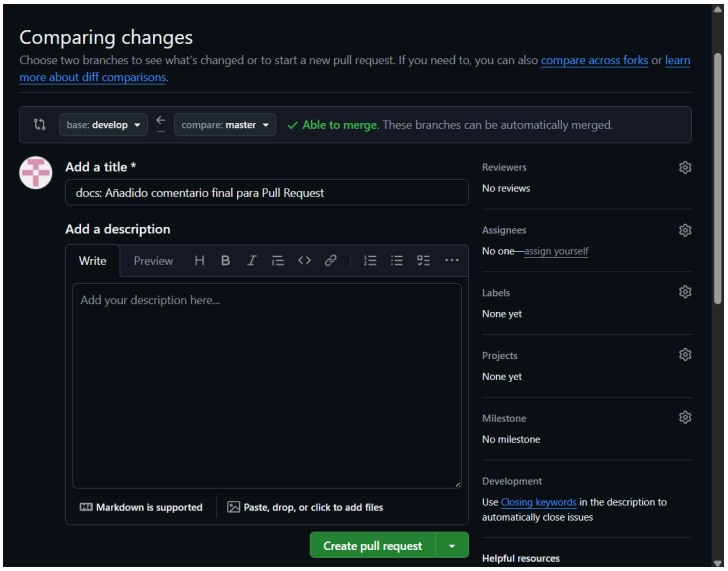


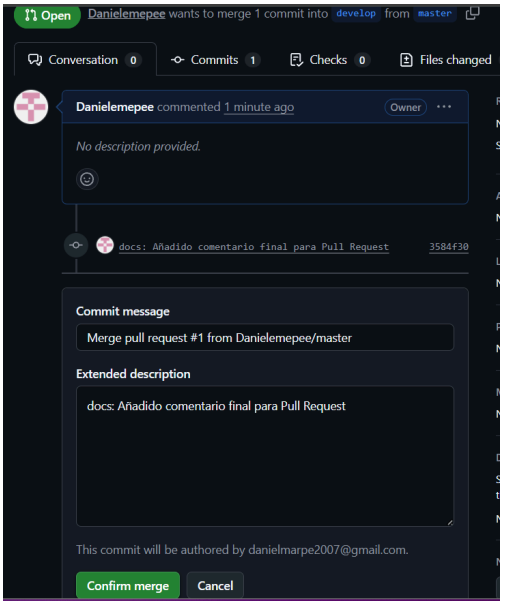
Después

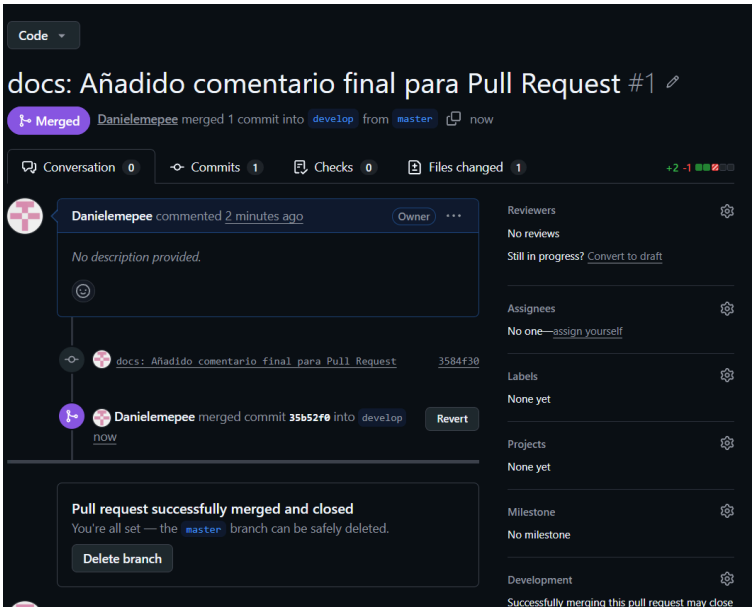
```java
        if (tamano > TAMANO_MAX_BYTES) {
            System.out.println("Archivo grande, comprimiendo...")
        }

        String extension = obtenerExtension(ruta);
        System.out.println("Extensión: " + extension);
    }
}

private String obtenerExtension(String ruta) {
```

3. Pull Request

1.







3. Técnicas de Refactorización Aplicadas (Eclipse)

A. Extract Method (Extracción de Método)

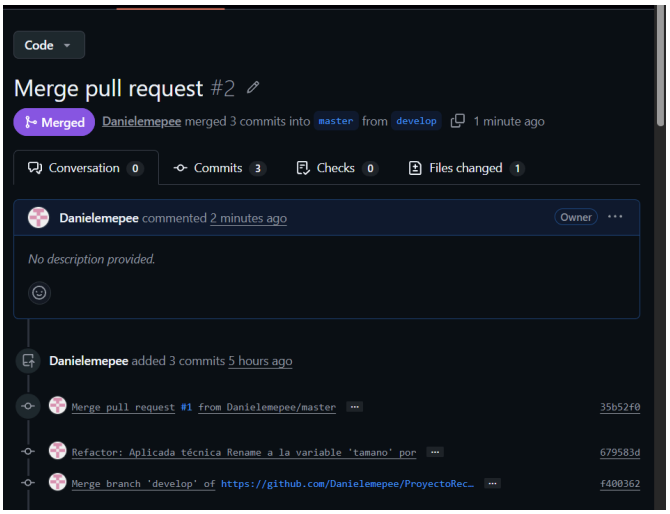
- **Descripción:** Se ha extraído la lógica de obtención de la extensión del archivo a un método privado.

B. Extract Constant (Extracción de Constantes)

Descripción: Se ha eliminado el "número mágico" `1048576` y se ha sustituido por una constante con nombre semántico.

```diff
src/com/recuperacion/FileManager.java                                  +7 -5
@@ -4,21 +4,23 @@
 4  4
 5  5   public class FileManager {
 6  6
 7     -    public void procesarArchivo(String ruta) {
       7  +    private static final int TAMANO_MAX_BYTES = 1024 * 1024;
       8  +
       9  +    public void procesarArchivo(String ruta) {
 8  10          File archivo = new File(ruta);
 9  11          if (archivo.exists()) {
10  12              System.out.println("Archivo encontrado: " + archivo.getName());
11  13              long tamano = archivo.length();
12  14              System.out.println("Tamaño: " + tamano + " bytes");
13  15
14     -            if (tamano > 1024 * 1024) {
       16 +         if (tamano > TAMANO_MAX_BYTES) {
15  17                  System.out.println("Archivo grande, comprimiendo...");
16  18              }
17  19
18  20              String extension = "";
19     -            int i = ruta.lastIndexOf('.');
20     -            if (i > 0) {
21     -                extension = ruta.substring(i + 1);
       21 +         int posicionPunto = ruta.lastIndexOf('.');
       22 +         if (posicionPunto > 0) {
       23 +             extension = ruta.substring(posicionPunto + 1);
22  24              }
23  25              System.out.println("Extensión: " + extension);
24  26          }
```

C. Rename

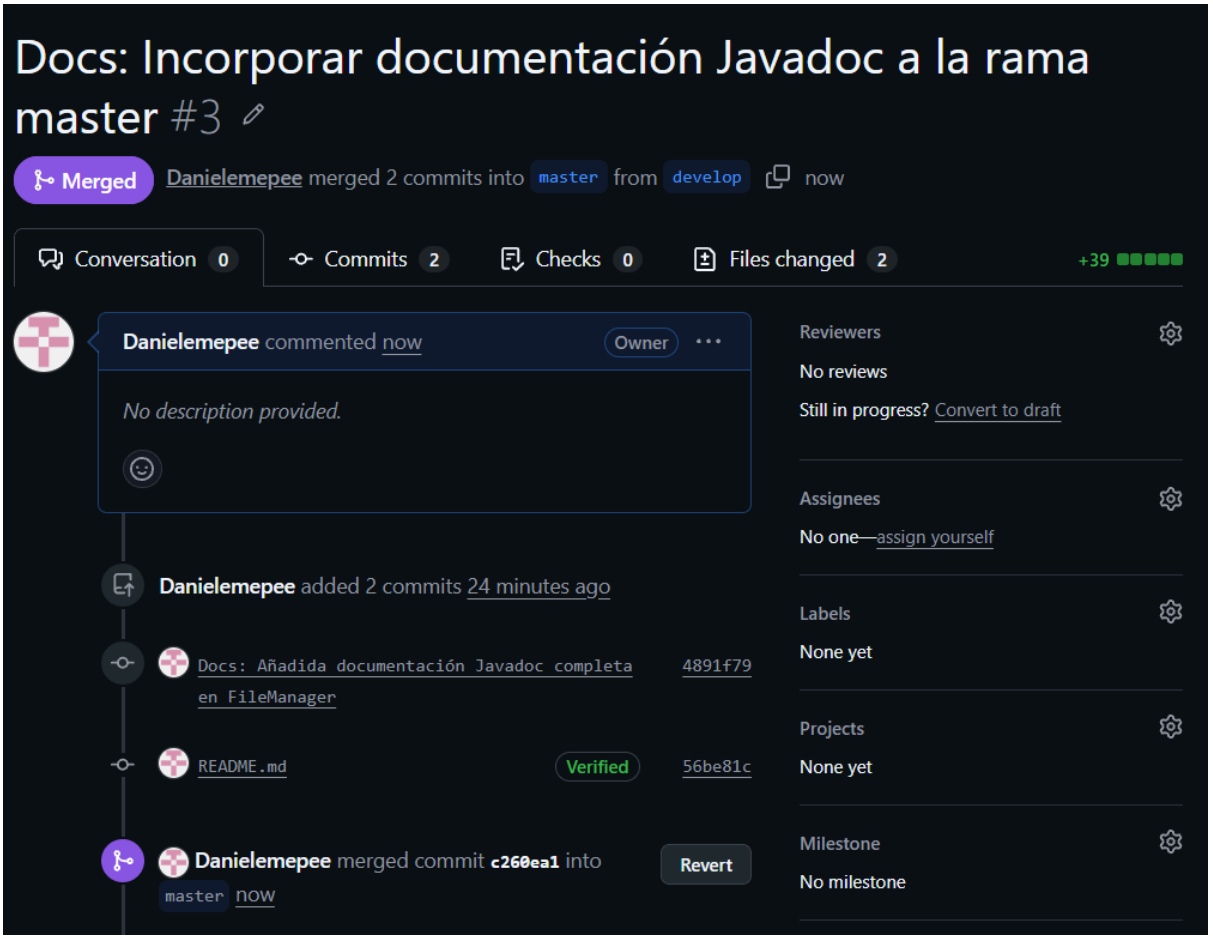



```diff
src/com/recuperacion/FileManager.java                                  +3 -3   Viewed
@@ -10,10 +10,10 @@ public void procesarArchivo(String ruta) {
10  10          File archivo = new File(ruta);
11  11          if (archivo.exists()) {
12  12              System.out.println("Archivo encontrado: " + archivo.getName());
13     -            long tamano = archivo.length();
14     -            System.out.println("Tamaño: " + tamano + " bytes");
       13 +         long tamanoArchivoBytes = archivo.length();
       14 +         System.out.println("Tamaño: " + tamanoArchivoBytes + " bytes");
15  15
16     -            if (tamano > TAMANO_MAX_BYTES) {
       16 +         if (tamanoArchivoBytes > TAMANO_MAX_BYTES) {
17  17                  System.out.println("Archivo grande, comprimiendo...");
18  18              }
19  19
```

Comentarios javadoc

```java
import java.io.File;

/**
 * Clase encargada de la gestión y procesamiento de archivos.
 * @author Tu Nombre
 * @version 1.0
 */
public class FileManager {

    private static final int TAMANO_MAX_BYTES = 1024 * 1024;

    /**
     * Procesa los archivos de una lista validando su tamaño y extensión.
     */
    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamanoArchivoBytes = archivo.length();
            System.out.println("Tamaño: " + tamanoArchivoBytes + " bytes");

            if (tamanoArchivoBytes > TAMANO_MAX_BYTES) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = obtenerExtension(ruta);
            System.out.println("Extensión: " + extension);
        }
    }

    /**
     * Extrae la extensión de un archivo a partir de su nombre.
     * @param nombreArchivo Nombre completo del fichero (ej: "foto.jpg")
     * @return La extensión del archivo después del punto en minúsculas.
     */
    private String obtenerExtension(String ruta) {
        String extension = "";
        int posicionPunto = ruta.lastIndexOf('.');
        if (posicionPunto > 0) {
            extension = ruta.substring(posicionPunto + 1);
        }
        return extension;
    }
}
//Fin de la clase FileManager
```

Hago un pull Request para incorporar toda la documentación Javadoc a master



Creo el HTML

SEARCH Search

Package com.recuperacion

Class FileManager

java.lang.Object
com.recuperacion.FileManager

```
public class FileManager
extends Object
```

Clase encargada de la gestión y procesamiento de archivos.

Version:

1.0

Author:

Daniel

Constructor Summary

Constructors

Constructor	Description
`FileManager()`	

Method Summary

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method	Description
void	`procesarArchivo(String ruta)`	Procesa los archivos de una lista validando su tamaño y extensión.

Methods inherited from class java.lang.Object

`equals`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

FileManager

```
public FileManager()
```

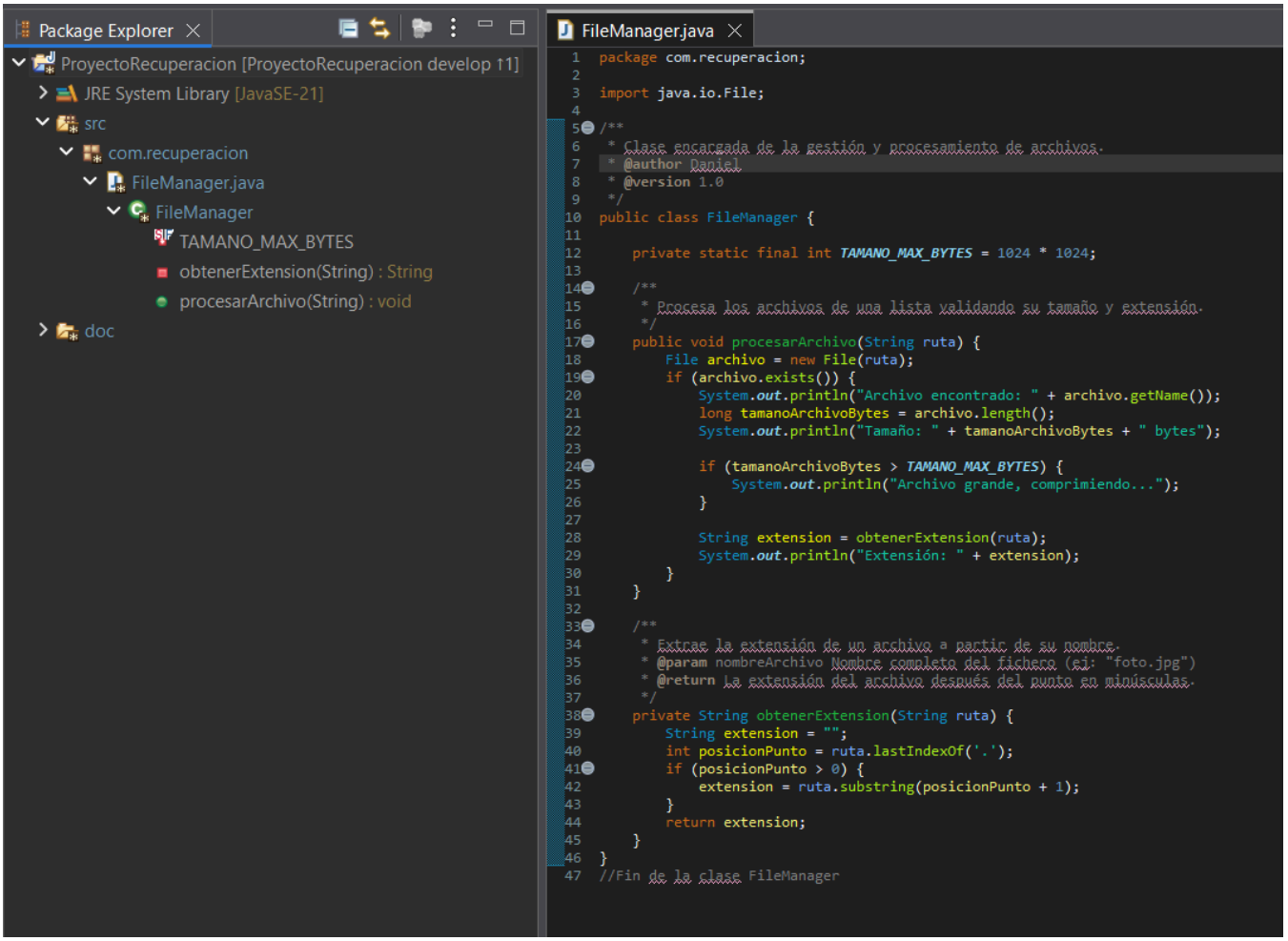
Method Details

procesarArchivo

```
public void procesarArchivo(String ruta)
```

Procesa los archivos de una lista validando su tamaño y extensión.

Código final




```java
package com.recuperacion;

import java.io.File;

/**
 * Clase encargada de la gestión y procesamiento de archivos.
 * @author Daniel
 * @version 1.0
 */
public class FileManager {

    private static final int TAMANO_MAX_BYTES = 1024 * 1024;

    /**
     * Procesa los archivos de una lista validando su tamaño y extensión.
     */
    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamanoArchivoBytes = archivo.length();
            System.out.println("Tamaño: " + tamanoArchivoBytes + " bytes");

            if (tamanoArchivoBytes > TAMANO_MAX_BYTES) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = obtenerExtension(ruta);
            System.out.println("Extensión: " + extension);
        }
    }

    /**
     * Extrae la extensión de un archivo a partir de su nombre.
     * @param nombreArchivo Nombre completo del fichero (ej: "foto.jpg")
     * @return La extensión del archivo después del punto en minúsculas.
     */
    private String obtenerExtension(String ruta) {
        String extension = "";
        int posicionPunto = ruta.lastIndexOf('.');
        if (posicionPunto > 0) {
            extension = ruta.substring(posicionPunto + 1);
        }
        return extension;
    }
}
//Fin de la clase FileManager
```